TAGORE ENGINEERING COLLEGE

RATHINAMANGALAM, CHENNAI- 600127

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS25C08-DATA STRUCTURES LABORATORY

Name : _____

Reg. No : _____

Branch : _____

Year : _____

Semester : _____

Anna University:: Chennai

Regulation 2025

TAGORE ENGINEERING COLLEGE

RATHINAMANGALAM, CHENNAI-600127

LABORATORY RECORD

UNIVERSITY REGISTER NO. | |

Certified that this is the bonafide record of work done by

Mr. /Ms. _____of _____

Department in the _____laboratory

and submitted for the University Practical Examination conducted on_____

at TAGORE ENGINEERING COLLEGE, CHENNAI-127.

Record Mark: _____

Lab In-charge **Principal** **Head of the Department**

Internal Examiner **External Examiner**

VISION

To create an environment which is conducive to produce competent Computer Science Engineers through quality education and research-oriented education and equip them for the needs of the industry and society.

MISSION

The Department strives to contribute to the expansion of knowledge in the discipline of Computer Science and Engineering by

- Adopting an efficient teaching learning process in concurrence with increasing industrial demands.
- Ensuring technical proficiency, facilitating to pursue higher studies and carry out Research & Development activities.
- Developing problem solving and analytical skills with deep knowledge in thorough understanding of basic sciences
- and Computer Science Engineering.
- Infusing managerial and entrepreneurship skills to become ethical, socially responsible, and competitive professionals.

PROGRAM SPECIFIC OBJECTIVES (PSOs)

1. Exhibit design and programming skills to build and automate business solutions using cutting edge technologies
2. Strong theoretical foundation leading to excellence and excitement towards research, to provide elegant solutions to complex problems.
3. Ability to work effectively with various engineering fields as a team to design, build and develop system applications.

PROGRAM OUTCOMES Pos

Engineering Graduates will be able to:

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change

COURSE OUTCOMES:

At the end of this course, the students will be able to:

CO1: Describe the operating systems concepts in managing computer hardware and software resources.

CO2: Analyze operating system components and file systems .

CO3: Evaluate operating system algorithms in different computing environments.

CO4: Design operating system solutions for managing system resources in real-world scenarios.

CO5: Develop the ability to adapt to emerging trends, tools, and technologies.

GENERAL INSTRUCTIONS FOR LABORATORY CLASSES

- Students must wear ID card and shoes before entering the lab.
- Boys should wear formal shirt, pant and should “tuck in” the shirts.
- You must shutdown the system, switch off the power supply and arrange the chairs properly before leaving the lab.
- Maintain Observation Notebook and Lab Record neatly and complete them then and there after completion of each exercise.
- Maintain discipline in the lab and be silent throughout the lab session.

INDEX

EX.NO	DATE	NAME OF THE EXPERIMENTS	MARK	SIGNATURE
1a		Implementation of Single dimensional Arrays		
1b		Implementation of Multidimensional Arrays		
2a		Implementation of Singly Linked List Operations		
2b		Implementation of Doubly Linked List Operations		
2c		Implementation of Circular Linked List Operations		
3a		Implementation of String Reverse Operations		
3b		Implementation of Expression Evaluation		
4a		Implementation of Circular Queue Operations		
4b		Implementation of Priority Queue Operations		
5		Implementation of Tree Traversal Operations		
6		Implementation of AVL Tree Rotations		
7		Implementation of Query and Update Operations on Balanced BSTs		
8a		Implementation of Quick Sort Algorithm		
8b		Implementation of Heap Sort Algorithm		
9a		Implementation of Binary Search Algorithm		
9b		Implementation of Hashing Techniques		
10a		Implementation of Breadth First Search(BFS)		
10b		Implementation of Depth First Search(DFS)		

11a		Implementation of Minimum Spanning Tree–Prim's Algorithm		
11b		Implementation of Minimum Spanning Tree– Kruskal's Algorithm		
11c		Implementation of Shortest Path Algorithm– Dijkstra's Algorithm		

EX.NO: 1a	SINGLE AND MULTIDIMENSIONAL ARRAYS
DATE:	IMPLEMENTATION OF SINGLE DIMENSIONAL ARRAYS

Aim

To write a C++ program to perform operations on a Single Dimensional Array and display its elements.

Algorithm

1. Start the program.
2. Declare an array and variables.
3. Read the number of elements (n).
4. Read the array elements from the user.
5. Display the entered array elements.
6. Stop the program.

Program

```
#include <iostream>
using namespace std;

int main()
{
   int a[100], n, i;

   cout << "Enter the number of elements: ";
   cin >> n;

   cout << "Enter the array elements:\n";
   for (i = 0; i < n; i++)
   {
      cin >> a[i];
   }

   cout << "Array elements are:\n";
   for (i = 0; i < n; i++)
   {
      cout << a[i] << " ";
   }
   cout << endl;

   return 0;
}
```

Output

```
Enter the number of elements: 5
Enter the array elements: 10 20
30 40 50 Array elements are:
10 20 30 40 50
```

Result

Thus, the C++ program to create and display a Single Dimensional Array was successfully executed and the output was verified.

EX.NO: 1b	SINGLE AND MULTIDIMENSIONAL ARRAYS
DATE:	IMPLEMENTATION OF MULTIDIMENSIONAL ARRAYS

Aim

To write a C++ program to create, read, and display the elements of a two-dimensional (multi-dimensional) array.

Algorithm

1. Start the program.
2. Declare a two-dimensional array.
3. Read the number of rows and columns.
4. Input the matrix elements using nested loops.
5. Display the matrix elements using nested loops.
6. Stop the program.

Program

```
#include <iostream>
using namespace std;

int main()
{
   int a[10][10], r, c, i, j;

   cout << "Enter number of rows and columns: ";
   cin >> r >> c;

   cout << "Enter matrix elements:\n";
   for (i = 0; i < r; i++)
   {
      for (j = 0; j < c; j++)
      {
         cin >> a[i][j];
      }
   }

   cout << "\nMatrix Elements are:\n";
   for (i = 0; i < r; i++)
   {
      for (j = 0; j < c; j++)
      {
         cout << a[i][j] << "\t";
      }
      cout << endl;
   }
   return 0;
}
```

Output

```
Enter number of rows and columns: 3 3

Enter matrix elements:
1 2 3
4 5 6
7 8 9
Matrix Elements are:
1 2 3
4 5 6
7 8 9
```

Result

Thus, the C++ program to create and display a Multi-Dimensional (Two-Dimensional) Array was successfully executed and the output was verified.

EX.NO: 2 a	SINGLY, DOUBLY AND CIRCULAR LINKED LISTS
DATE:	IMPLEMENTATION OF SINGLY LINKED LISTS

Aim

To implement a Singly Linked List in C++ and perform basic operations such as insertion, display, and deletion of nodes.

Algorithm

Insertion at End

1. Start.
2. Create a new node.
3. Enter the data for the node.
4. If the list is empty, make the new node as the head.
5. Otherwise, traverse to the last node.
6. Link the last node to the new node.
7. Stop.

Display

1. Start.
2. Check whether the list is empty.
3. If empty, display "List is Empty".
4. Otherwise, traverse from the head node.
5. Display each node's data.
6. Stop.

Deletion at Beginning

1. Start.
2. Check whether the list is empty.
3. If empty, display "List is Empty".
4. Otherwise, store the head node in a temporary pointer.
5. Move the head to the next node.
6. Delete the temporary node.
7. Stop.

Program

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *next;
};

Node *head = NULL;

// Insert at End
```

```
void insert(int value)
{
   Node *newNode = new Node();
   newNode->data = value;
   newNode->next = NULL;

   if (head == NULL)
   {
      head = newNode;
   }
   else
   {
      Node *temp = head;
      while (temp->next != NULL)
      {
         temp = temp->next;
      }
      temp->next = newNode;
   }
}

// Display List
void display()
{
   Node *temp = head;

   if (temp == NULL)
   {
      cout << "List is Empty" << endl;
      return;
   }

   cout << "Linked List: ";
   while (temp != NULL)
   {
      cout << temp->data << " ";
      temp = temp->next;
   }
   cout << endl;
}

// Delete at Beginning
void deleteBegin()
{
   if (head == NULL)
   {
      cout << "List is Empty" << endl;
      return;
   }
```

```
    Node *temp = head;
    head = head->next;
    delete temp;
    cout << "First node deleted successfully." << endl;
}

int main()
{
    insert(10);
    insert(20);
    insert(30);
    display();

    deleteBegin();
    display();

    return 0;
}
```

Output

Linked List: 10 20 30
First node deleted successfully. Linked
List: 20 30

Result

Thus, the Singly Linked List was successfully implemented in C++ and the operations of insertion, display, and deletion were performed successfully.

EX.NO: 2 b	SINGLY, DOUBLY AND CIRCULAR LINKED LISTS
DATE:	IMPLEMENTATION OF DOUBLY LINKED LISTS

Aim

To implement a Doubly Linked List in C++ and perform insertion, display, and deletion operations.

Algorithm

Insertion at End

1. Start.
2. Create a new node with data.
3. Set next and prev of the new node to NULL.
4. If the list is empty, make the new node as the head.
5. Otherwise, traverse to the last node.
6. Link the last node's next to the new node.
7. Set the new node's prev to the last node.
8. Stop.

Display

1. Start.
2. Check if the list is empty.
3. If empty, display "List is Empty".
4. Otherwise, traverse from the head node.
5. Display each node's data.
6. Stop.

Deletion at Beginning

1. Start.
2. Check if the list is empty.
3. If empty, display "List is Empty".
4. Store the head node in a temporary pointer.
5. Move head to the next node.
6. Set the new head node's prev to NULL.
7. Delete the temporary node.
8. Stop.

Program

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *prev;
   Node *next;
};

Node *head = NULL;
```

```
// Insert at End
void insert(int value)
{
   Node *newNode = new Node();
   newNode->data = value;
   newNode->prev = NULL;
   newNode->next = NULL;

   if (head == NULL)
   {
      head = newNode;
   }
   else
   {
      Node *temp = head;
      while (temp->next != NULL)
      {
         temp = temp->next;
      }
      temp->next = newNode;
      newNode->prev = temp;
   }
}

// Display List
void display()
{
   Node *temp = head;

   if (temp == NULL)
   {
      cout << "List is Empty" << endl;
      return;
   }

   cout << "Doubly Linked List: ";
   while (temp != NULL)
   {
      cout << temp->data << " ";
      temp = temp->next;
   }
   cout << endl;
}

// Delete First Node
void deleteBegin()
{
   if (head == NULL)
```

```
    {
        cout << "List is Empty" << endl;
        return;
    }

    Node *temp = head;
    head = head->next;

    if (head != NULL)
        head->prev = NULL;

    delete temp;
    cout << "First node deleted successfully." << endl;
}

int main()
{
    insert(10);
    insert(20);
    insert(30);
    display();

    deleteBegin();
    display();

    return 0;
}
```

Output

Doubly Linked List: 10 20 30 First
node deleted successfully. Doubly
Linked List: 20 30

Result

Thus, the Doubly Linked List was successfully implemented in C++ and the operations of insertion, display, and deletion were performed successfully.

EX.NO: 2 c	SINGLY, DOUBLY AND CIRCULAR LINKED LISTS
DATE:	IMPLEMENTATION OF CIRCULAR LINKED LISTS

Aim

To implement a Circular Linked List in C++ and perform all basic operations such as insertion, deletion, searching, and display.

Algorithm

1. **Insert at Beginning** 1. Create a new node.
2. If the list is empty:
   - Make the new node point to itself.
   - Set it as head.
3. Otherwise:
   - Traverse to the last node. ○ Link the new node to head.
   - Link the last node to the new node.
   - Update head to the new node.
4. Stop.

2. Insert at End

1. Create a new node.
2. If the list is empty:
   - Make the new node point to itself.
   - Set it as head.
3. Otherwise:
   - Traverse to the last node.
   - Link the last node to the new node.
   - Link the new node to head.
4. Stop.

3. Delete at Beginning

1. If the list is empty, display message.
2. If only one node exists:
   - Delete the node. ○ Set head to NULL.
3. Otherwise:
   - Find the last node. ○ Move head to the next node. ○ Link last node to new head.
   - Delete old head.
4. Stop.

4. Delete at End

1. If the list is empty, display message.
2. If only one node exists:
   - Delete it and set head to NULL.
3. Otherwise:
   - Traverse to the second-last node.
   - Delete the last node. ○ Link second-last node to head.

4. Stop.

5. Search an Element

1. Start from head.
2. Traverse each node.
3. Compare node data with search key.
4. If found, display position.
5. Otherwise, display "Element Not Found".
6. Stop.

6. Display

1. Check if list is empty.
2. If empty, display message.
3. Otherwise, traverse from head until head is reached again.
4. Display all elements.
5. Stop.

Program

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *next;
};

Node *head = NULL;

// Insert at Beginning
void insertBegin(int value)
{
   Node *newNode = new Node();
   newNode->data = value;

   if (head == NULL)
   {
      head = newNode;
      newNode->next = head;
   }
   else
   {
      Node *temp = head;
      while (temp->next != head)
         temp = temp->next;

      newNode->next = head;
      temp->next = newNode;
      head = newNode;
   }
```

```
}

// Insert at End
void insertEnd(int value)
{
   Node *newNode = new Node();
   newNode->data = value;

   if (head == NULL)
   {
      head = newNode;
      newNode->next = head;
   }
   else
   {
      Node *temp = head;
      while (temp->next != head)
         temp = temp->next;

      temp->next = newNode;
      newNode->next = head;
   }
}

// Delete at Beginning
void deleteBegin()
{
   if (head == NULL)
   {
      cout << "List is Empty\n";
      return;
   }

   if (head->next == head)
   {
      delete head;
      head = NULL;
   }
   else
   {
      Node *temp = head;
      Node *last = head;

      while (last->next != head)
         last = last->next;

      head = head->next;
      last->next = head;
      delete temp;
```

```
    }
}

// Delete at End
void deleteEnd()
{
    if (head == NULL)
    {
        cout << "List is Empty\n";
        return;
    }

    if (head->next == head)
    {
        delete head;
        head = NULL;
    }
    else
    {
        Node *temp = head;
        while (temp->next->next != head)
            temp = temp->next;

        Node *last = temp->next;
        temp->next = head;
        delete last;
    }
}

// Search Element
void search(int key)
{
    if (head == NULL)
    {
        cout << "List is Empty\n";
        return;
    }

    Node *temp = head;
    int pos = 1;

    do
    {
        if (temp->data == key)
        {
            cout << "Element found at position " << pos << endl;
            return;
        }
        temp = temp->next;
```

```
        pos++;
    } while (temp != head);

    cout << "Element Not Found\n";
}

// Display
void display()
{
    if (head == NULL)
    {
        cout << "List is Empty\n";
        return;
    }

    Node *temp = head;
    cout << "Circular Linked List: ";

    do
    {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != head);

    cout << endl;
}

int main()
{
    insertBegin(10);
    insertEnd(20);
    insertEnd(30);
    insertEnd(40);
    display();

    search(30);

    deleteBegin();
    display();

    deleteEnd();
    display();

    return 0;
}
```

Output

Circular Linked List: 10 20 30 40
Element found at position 3
Circular Linked List: 20 30 40
Circular Linked List: 20 30

Result

Thus, the Circular Linked List was successfully implemented in C++ and the operations such as insertion at beginning, insertion at end, deletion at beginning, deletion at end, searching, and display were performed successfully.

EX.NO: 3a	STRING REVERSE OPERATIONS AND EXPRESSION EVALUATION
DATE:	IMPLEMENTATION OF STRING REVERSE OPERATIONS

Aim

To write a C++ program to reverse a given string and display the reversed string.

Algorithm

1. Start.
2. Declare a string variable.
3. Read the input string from the user.
4. Find the length of the string.
5. Traverse the string from the last character to the first character.
6. Print each character in reverse order.
7. Stop.

Program

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
   string str;

   cout << "Enter a String: ";
   getline(cin, str);

   cout << "Reversed String: ";
   for (int i = str.length() - 1; i >= 0; i--)
   {
      cout << str[i];
   }
   cout << endl;

   return 0;
}
```

Output

Enter a String: DATA STRUCTURE
Reversed String: ERUTCURTS ATAD

Result

Thus, the C++ program to reverse a given string was implemented and executed successfully.

EX.NO: 3b	STRING REVERSE OPERATIONS AND EXPRESSION EVALUATION
DATE:	IMPLEMENTATION OF EXPRESSION EVALUATION

Aim

To write a C++ program to perform Expression Evaluation using Stack.

Algorithm

1. Start.
2. Read a postfix expression.
3. Traverse each symbol in the expression.
4. If the symbol is an operand, push it onto the stack.
5. If the symbol is an operator: ○ Pop two operands. ○ Perform the operation.
   ○ Push the result back onto the stack.
6. Repeat until the expression ends.
7. The final value in the stack is the result.
8. Display the result.
9. Stop.

Program

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main()
{
   string postfix;
   stack<int> st;

   cout << "Enter Postfix Expression: ";
   cin >> postfix;

   for (char ch : postfix)
   {
      if (isdigit(ch))
      {
         st.push(ch - '0');
      }
      else
      {
         int b = st.top();
         st.pop();
         int a = st.top();
         st.pop();
```

```
            switch (ch)
            {
                case '+':
                    st.push(a + b);
                    break;
                case '-':
                    st.push(a - b);
                    break;
                case '*':
                    st.push(a * b);
                    break;
                case '/':
                    st.push(a / b);
                    break;
            }
        }
    }

    cout << "Result = " << st.top() << endl;

    return 0;
}
```

Output

Enter Postfix Expression: 23+5*
Result = 25

Evaluation

Expression:
23+5*
= (2+3)*5
= 5*5
= 25

Result

Thus, the C++ programs for String Reverse Operation and Postfix Expression Evaluation using Stack were implemented and executed successfully.

EX.NO: 4 a	CIRCULAR QUEUE AND PRIORITY QUEUE
DATE:	IMPLEMENTATION OF CIRCULAR QUEUE

Aim

To implement a Circular Queue in C++ and perform operations such as Enqueue, Dequeue, Display, and Peek.

Algorithm

Enqueue Operation

1. Start.
2. Check whether the queue is full.
3. If full, display "Queue Overflow".
4. If the queue is empty, set front = rear = 0.
5. Otherwise, update rear = (rear + 1) % size.
6. Insert the element at rear.
7. Stop.

Dequeue Operation

1. Start.
2. Check whether the queue is empty.
3. If empty, display "Queue Underflow".
4. Store the element at front.
5. If front == rear, set front = rear = -1.
6. Otherwise, update front = (front + 1) % size.
7. Display the deleted element.
8. Stop.

Display Operation

1. Start.
2. Check whether the queue is empty.
3. If empty, display "Queue is Empty".
4. Traverse from front to rear in circular manner.
5. Display all elements.
6. Stop.

Peek Operation

1. Start.
2. Check whether the queue is empty.
3. If empty, display "Queue is Empty".
4. Display the element at front.
5. Stop

Program

```
#include <iostream>
using namespace std;

#define SIZE 5
```

```
class CircularQueue
{
    int queue[SIZE];
    int front, rear;

public:
    CircularQueue()
    {
        front = rear = -1;
    }

    void enqueue(int value)
    {
        if ((rear + 1) % SIZE == front)
        {
            cout << "Queue Overflow" << endl;
            return;
        }

        if (front == -1)
            front = rear = 0;
        else
            rear = (rear + 1) % SIZE;

        queue[rear] = value;
        cout << value << " inserted" << endl;
    }

    void dequeue()
    {
        if (front == -1)
        {
            cout << "Queue Underflow" << endl;
            return;
        }

        cout << queue[front] << " deleted" << endl;

        if (front == rear)
            front = rear = -1;
        else
            front = (front + 1) % SIZE;
    }

    void peek()
    {
        if (front == -1)
        {
            cout << "Queue is Empty" << endl;
```

```
            return;
        }

        cout << "Front Element = " << queue[front] << endl;
    }

    void display()
    {
        if (front == -1)
        {
            cout << "Queue is Empty" << endl;
            return;
        }

        cout << "Circular Queue: ";
        int i = front;

        while (true)
        {
            cout << queue[i] << " ";
            if (i == rear)
                break;
            i = (i + 1) % SIZE;
        }
        cout << endl;
    }
};

int main()
{
    CircularQueue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.enqueue(40);
    q.display();

    q.peek();

    q.dequeue();
    q.display();

    return 0;
}
```

Output

```
10 inserted
20 inserted
30 inserted
40 inserted

Circular Queue: 10 20 30 40
Front Element = 10
10 deleted

Circular Queue: 20 30 40
```

Result

Thus, the Circular Queue was successfully implemented in C++ and the operations Enqueue, Dequeue, Display, and Peek were performed successfully.

EX.NO: 4 b	CIRCULAR QUEUE AND PRIORITY QUEUE
DATE:	IMPLEMENTATION OF PRIORITY QUEUE

Aim

To implement a Priority Queue in C++ and perform operations such as insertion, deletion, and display based on priority.

Algorithm

Insertion (Enqueue)

1. Start.
2. Read the element and its priority.
3. Insert the element into the queue according to its priority.
4. Higher-priority elements are placed before lower-priority elements.
5. Stop.

Deletion (Dequeue)

1. Start.
2. Check whether the queue is empty.
3. If empty, display "Queue Underflow".
4. Remove the element with the highest priority.
5. Display the deleted element.
6. Stop.

Display

1. Start.
2. Check whether the queue is empty.
3. If empty, display "Queue is Empty".
4. Traverse the queue from front to rear.
5. Display elements along with their priorities.
6. Stop.

Program

```
#include <iostream>
using namespace std;

#define SIZE 5

class PriorityQueue
{
   int data[SIZE];
   int priority[SIZE];
   int count;

public:
   PriorityQueue()
   {
      count = 0;
   }
```

```
void enqueue(int value, int pr)
{
   if (count == SIZE)
   {
      cout << "Queue Overflow" << endl;
      return;
   }

   int i = count - 1;
   while (i >= 0 && priority[i] < pr)
   {
      data[i + 1] = data[i];
      priority[i + 1] = priority[i];
      i--;
   }

   data[i + 1] = value;
   priority[i + 1] = pr;
   count++;

   cout << value << " inserted with priority " << pr << endl;
}

void dequeue()
{
   if (count == 0)
   {
      cout << "Queue Underflow" << endl;
      return;
   }

   cout << data[0] << " deleted (Highest Priority)" << endl;

   for (int i = 0; i < count - 1; i++)
   {
      data[i] = data[i + 1];
      priority[i] = priority[i + 1];
   }
   count--;
}

void display()
{
   if (count == 0)
   {
      cout << "Queue is Empty" << endl;
      return;
   }
```

```
        cout << "\nPriority Queue:\n";
        for (int i = 0; i < count; i++)
        {
            cout << "Element: " << data[i]
                 << " | Priority: " << priority[i] << endl;
        }
    }
};

int main()
{
    PriorityQueue pq;

    pq.enqueue(10, 2);
    pq.enqueue(20, 5);
    pq.enqueue(30, 1);
    pq.enqueue(40, 4);
    pq.display();

    pq.dequeue();
    pq.display();

    return 0;
}
```

Output

```
10 inserted with priority 2
20 inserted with priority 5
30 inserted with priority 1
40 inserted with priority 4

Priority Queue:
Element: 20 Priority: 5
Element: 40 Priority: 4
Element: 10 Priority: 2
Element: 30 Priority: 1 20
deleted (Highest Priority)
Priority Queue:
Element: 40 Priority: 4
Element: 10 Priority: 2
Element: 30 Priority: 1
```

Result

Thus, the Priority Queue was successfully implemented in C++ and the operations of insertion, deletion, and display were performed successfully.

EX.NO: 5	TRAVERSAL OPERATION
DATE:	

Aim

To implement Binary Tree Traversal Operations in C++ and perform Preorder, Inorder, and Postorder Traversals.

Algorithm

Preorder Traversal (Root → Left → Right)

1. Visit the root node.
2. Traverse the left subtree in preorder.
3. Traverse the right subtree in preorder.
4. Stop.

Inorder Traversal (Left → Root → Right)

1. Traverse the left subtree in inorder.
2. Visit the root node.
3. Traverse the right subtree in inorder.
4. Stop.

Postorder Traversal (Left → Right → Root)

1. Traverse the left subtree in postorder.
2. Traverse the right subtree in postorder.
3. Visit the root node.
4. Stop.

Program

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *left;
   Node *right;
};

Node* createNode(int value)
{
   Node* newNode = new Node();
   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}

// Preorder Traversal
```

```
void preorder(Node* root)
{
   if (root != NULL)
   {
      cout << root->data << " ";
      preorder(root->left);
      preorder(root->right);
   }
}

// Inorder Traversal
void inorder(Node* root)
{
   if (root != NULL)
   {
      inorder(root->left);
      cout << root->data << " ";
      inorder(root->right);
   }
}

// Postorder Traversal
void postorder(Node* root)
{
   if (root != NULL)
   {
      postorder(root->left);
      postorder(root->right);
      cout << root->data << " ";
   }
}

int main()
{
   Node* root = createNode(10);
   root->left = createNode(20);
   root->right = createNode(30);
   root->left->left = createNode(40);
   root->left->right = createNode(50);
   cout << "Preorder Traversal: ";
   preorder(root);
   cout << "\nInorder Traversal: ";
   inorder(root);
   cout << "\nPostorder Traversal: ";
   postorder(root);
   cout << endl;
   return 0;
}
```

Output

Preorder Traversal: 10 20 40 50 30
Inorder Traversal: 40 20 50 10 30
Postorder Traversal: 40 50 20 30 10

Result

Thus, the Binary Tree Traversal Operations (Preorder, Inorder, and Postorder) were successfully implemented and executed in C++.

EX.NO: 6	AVL TREE ROTATIONS
DATE:	

Aim

To implement AVL Tree Rotations in C++ and perform balancing operations using LL, RR, LR, and RL Rotations.

Algorithm

AVL Insertion

1. Insert the node as in a Binary Search Tree.
2. Update the height of the ancestor node.
3. Calculate the balance factor.
4. If balance factor exceeds the limit:
   - LL Case → Right Rotation
   - RR Case → Left Rotation
   - LR Case → Left Rotation + Right Rotation
   - RL Case → Right Rotation + Left Rotation
5. Return the balanced tree.

Program

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *left;
   Node *right;
   int height;
};

int height(Node *N)
{
   if (N == NULL)
      return 0;
   return N->height;
}

int max(int a, int b)
{
   return (a > b) ? a : b;
}

Node* newNode(int data)
{
   Node* node = new Node();
```

```
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return node;
}

Node* rightRotate(Node *y)
{
    Node *x = y->left;
    Node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}
Node* leftRotate(Node *x)
{
    Node *y = x->right;
    Node *T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}
int getBalance(Node *N)
{
    if (N == NULL)
        return 0;
    return height(N->left) - height(N->right);
}
Node* insert(Node* node, int key)
{
    if (node == NULL)
        return newNode(key);
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));
```

```
    int balance = getBalance(node);

    // LL Rotation
    if (balance > 1 && key < node->left->data)
        return rightRotate(node);

    // RR Rotation
    if (balance < -1 && key > node->right->data)
        return leftRotate(node);

    // LR Rotation
    if (balance > 1 && key > node->left->data)
    {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    // RL Rotation
    if (balance < -1 && key < node->right->data)
    {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void preorder(Node *root)
{
    if (root != NULL)
    {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}
int main()
{
    Node *root = NULL;
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 10);

    cout << "Preorder Traversal of AVL Tree: ";
    preorder(root);
    cout << endl;

    return 0;
}
```

Output

Preorder Traversal of AVL Tree: 20
10 30

Result

Thus, the AVL Tree was successfully implemented in C++ and all balancing rotations (LL, RR, LR, RL) were performed to maintain a balanced tree.

EX.NO: 7	QUERY AND UPDATE OPERATIONS ON BALANCED BSTS
DATE:	

Aim

To implement Query and Update Operations on a Balanced Binary Search Tree (AVL Tree) in C++ and perform searching (query) and updating of node values while maintaining balance.

Algorithm

Query (Search) Operation 1.
Start from the root node.
2. Compare the key with the current node.
3. If equal, element is found.
4. If smaller, move to the left subtree.
5. If greater, move to the right subtree.
6. Repeat until the key is found or the tree becomes NULL.
7. Stop.

Update Operation

1. Search for the old value.
2. If found, delete the old value.
3. Insert the new value.
4. Rebalance the AVL Tree if necessary.
5. Display the updated tree.
6. Stop.

Program

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *left, *right;
};

Node* createNode(int value)
{
   Node* temp = new Node;
   temp->data = value;
   temp->left = temp->right = NULL;
   return temp;
}

Node* insert(Node* root, int value)
{
   if (root == NULL)
      return createNode(value);
```

```
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

bool search(Node* root, int key)
{
    if (root == NULL)
        return false;

    if (root->data == key)
        return true;

    if (key < root->data)
        return search(root->left, key);

    return search(root->right, key);
}

Node* findMin(Node* root)
{
    while (root->left != NULL)
        root = root->left;
    return root;
}

Node* deleteNode(Node* root, int key)
{
    if (root == NULL)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else
    {
        if (root->left == NULL)
        {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == NULL)
        {
```

```
            Node* temp = root->left;
            delete root;
            return temp;
        }

        Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void inorder(Node* root)
{
    if (root != NULL)
    {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

int main()
{
    Node* root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);

    cout << "Inorder Traversal: ";
    inorder(root);

    if (search(root, 30))
        cout << "\nElement 30 Found";
    else
        cout << "\nElement Not Found";

    root = deleteNode(root, 30);
    root = insert(root, 35);

    cout << "\nUpdated Tree: ";
    inorder(root);
    cout << endl;
    return 0;
}
```

Output

Inorder Traversal: 20 30 40 50 70

Element 30 Found

Updated Tree: 20 35 40 50 70

Result

Thus, the Query (Search) and Update operations on a Balanced Binary Search Tree were implemented successfully and executed correctly.

EX.NO: 8 a	QUICK AND HEAP SORT
DATE:	IMPLEMENTATION OF QUICK SORT

Aim

To write a C++ program to sort the given elements in ascending order using the Quick Sort technique.

Algorithm

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. Select the last element as the pivot.
5. Partition the array such that:
    - Elements smaller than the pivot are placed to its left.
    - Elements greater than the pivot are placed to its right.
6. Recursively apply Quick Sort to the left subarray.
7. Recursively apply Quick Sort to the right subarray.
8. Display the sorted array.
9. Stop.

Program

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high)
{
   int pivot = arr[high];
   int i = low - 1;

   for (int j = low; j < high; j++)
   {
      if (arr[j] < pivot)
      {
         i++;
         swap(arr[i], arr[j]);
      }
   }
   swap(arr[i + 1], arr[high]);
   return (i + 1);
}

void quickSort(int arr[], int low, int high)
{
   if (low < high)
   {
      int pi = partition(arr, low, high);
      quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
      cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

Output

Enter number of elements: 6
Enter elements: 45 12 78 34 23 10 Sorted
array: 10 12 23 34 45 78

Result

Thus, the C++ program for sorting elements using the Quick Sort algorithm was implemented and executed successfully.

EX.NO: 8 b	QUICK AND HEAP SORT
DATE:	IMPLEMENTATION OF HEAP SORT

Aim

To write a C++ program to sort the given elements in ascending order using the Heap Sort technique.

Algorithm

Heap Sort Algorithm

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. Build a Max Heap from the given array.
5. Swap the root element (largest element) with the last element.
6. Reduce the heap size by one.
7. Heapify the root node to maintain the Max Heap property.
8. Repeat Steps 5–7 until the heap size becomes 1.
9. Display the sorted array.
10. Stop.

Heapify Algorithm

1. Assume the current node as the largest.
2. Find the left child (2*i + 1) and right child (2*i + 2).
3. Compare the current node with its children.
4. If any child is larger, swap it with the current node.
5. Recursively heapify the affected subtree.

Program

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < n && arr[left] > arr[largest])
      largest = left;

   if (right < n && arr[right] > arr[largest])
      largest = right;

   if (largest != i)
   {
      swap(arr[i], arr[largest]);
      heapify(arr, n, largest);
```

```
    }
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

Output

Enter number of elements: 7
Enter elements: 45 20 15 10 30 50 25

Sorted Array: 10 15 20 25 30 45 50

Result

Thus, the C++ program for sorting elements using the Heap Sort algorithm was implemented and executed successfully.

EX.NO: 9 a	BINARY SEARCH AND HASHING
DATE:	IMPLEMENTATION OF BINARY SEARCH

Aim

To write a C++ program to search for a given element in a sorted array using the Binary Search technique.

Algorithm

1. Start.
2. Read the number of elements n.
3. Read the sorted array elements.
4. Read the element to be searched (key).
5. Initialize:
    - low = 0 ○ high = n - 1
6. Repeat while low <= high:
    - Find the middle position: mid = (low + high) / 2
    - If arr[mid] == key, display the position and stop.
    - If key < arr[mid], set high = mid - 1.
    - Otherwise, set low = mid + 1.
7. If the element is not found, display a message.
8. Stop.

Program

```
#include <iostream>
using namespace std;

int main()
{
   int n, key;

   cout << "Enter number of elements: ";
   cin >> n;

   int arr[n];

   cout << "Enter sorted elements: ";
   for (int i = 0; i < n; i++)
      cin >> arr[i];

   cout << "Enter element to search: ";
   cin >> key;

   int low = 0, high = n - 1;
   int mid;
   bool found = false;
```

```
    while (low <= high)
    {
        mid = (low + high) / 2;

        if (arr[mid] == key)
        {
            cout << "Element found at position " << mid + 1;
            found = true;
            break;
        }
        else if (key < arr[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    if (!found)
        cout << "Element not found";

    cout << endl;
    return 0;
}
```

Output

Output 1 (Element Found)
Enter number of elements: 6
Enter sorted elements: 10 20 30 40 50 60
Enter element to search: 40

Element found at position 4

Output 2 (Element Not Found)
Enter number of elements: 6
Enter sorted elements: 10 20 30 40 50 60
Enter element to search: 25
Element not found

Result

Thus, the C++ program for searching an element using the Binary Search algorithm was implemented and executed successfully.

EX.NO: 9 b	BINARY SEARCH AND HASHING
DATE:	IMPLEMENTATION OF HASHING

Aim

To write a C++ program to implement Hashing using a Hash Table with Linear Probing for collision resolution.

Algorithm

Insertion 1.

Start.

2. Read the size of the hash table.
3. Read the key to be inserted.
4. Compute the hash index: index = key % size
5. If the location is empty, insert the key.
6. Otherwise, move sequentially to the next position until an empty slot is found.
7. Repeat until all keys are inserted.
8. Stop.

Display

1. Traverse the hash table.
2. Display all elements along with their indices.
3. Stop.

Program

```
#include <iostream>
using namespace std;

class HashTable
{
   int table[10];
   int size;

public:
   HashTable(int s)
   {
      size = s;
      for (int i = 0; i < size; i++)
         table[i] = -1;
   }

   void insert(int key)
   {
      int index = key % size;

      while (table[index] != -1)
      {
         index = (index + 1) % size;
```

```
        }
        table[index] = key;
    }

    void display()
    {
        cout << "\nHash Table:\n";
        for (int i = 0; i < size; i++)
        {
            cout << i << " --> ";
            if (table[i] == -1)
                cout << "Empty";
            else
                cout << table[i];
            cout << endl;
        }
    }
};

int main()
{
    int n, key, count;

    cout << "Enter Hash Table Size: ";
    cin >> n;

    HashTable h(n);

    cout << "Enter Number of Keys: ";
    cin >> count;

    cout << "Enter Keys:\n";
    for (int i = 0; i < count; i++)
    {
        cin >> key;
        h.insert(key);
    }

    h.display();
    return 0;
}
```

Output

```
Enter Hash Table Size: 10
Enter Number of Keys: 5

Enter Keys:
25
35
45
20
30

Hash Table:

0 --> 20
1 --> 30
2 --> Empty
3 --> Empty
4 --> Empty
5 --> 25
6 --> 35
7 --> 45
8 --> Empty
9 --> Empty
```

Result

Thus, the C++ program for implementing Hashing using Linear Probing was executed successfully.

EX.NO: 10 a	BFS AND DFS ALGORITHMS
DATE:	IMPLEMENTATION OF BREADTH FIRST SEARCH (BFS) ALGORITHMS

Aim

To write a C++ program to traverse a graph using the Breadth First Search (BFS) algorithm.

Algorithm

BFS Traversal

1. Start.
2. Read the number of vertices in the graph.
3. Create the adjacency matrix or adjacency list.
4. Select the starting vertex.
5. Mark the starting vertex as visited.
6. Insert the starting vertex into the queue.
7. Repeat until the queue becomes empty:
    - Remove a vertex from the queue.
    - Display the vertex. ○ Visit all adjacent unvisited vertices.
    - Mark them as visited and insert them into the queue.
8. Stop.

Program

```
#include <iostream>
#include <queue>
using namespace std;

int main()
{
   int n, start;

   cout << "Enter number of vertices: ";
   cin >> n;

   int graph[20][20];

   cout << "Enter adjacency matrix:\n";
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         cin >> graph[i][j];
      }
   }

   cout << "Enter starting vertex: ";
   cin >> start;
```

```
    bool visited[20] = {false};
    queue<int> q;

    visited[start] = true;
    q.push(start);

    cout << "BFS Traversal: ";
    while (!q.empty())
    {
        int v = q.front();
        q.pop();
        cout << v << " ";

        for (int i = 0; i < n; i++)
        {
            if (graph[v][i] == 1 && !visited[i])
            {
                visited[i] = true;
                q.push(i);
            }
        }
    }
    cout << endl;

    return 0;
}
```

Output

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 1
1 0 0 0 0
0 1 0 0 0
0 1 0 0 0

Enter starting vertex: 0
BFS Traversal: 0 1 2 3 4
```

Result

Thus, the C++ program for Breadth First Search (BFS) graph traversal was implemented and executed successfully.

EX.NO: 10 b	BFS AND DFS ALGORITHMS
DATE:	IMPLEMENTATION OF DEPTH FIRST SEARCH (DFS) ALGORITHMS

Aim

To write a C++ program to traverse a graph using the Depth First Search (DFS) algorithm.

Algorithm

DFS Traversal

1. Start.
2. Read the number of vertices in the graph.
3. Create the adjacency matrix.
4. Read the starting vertex.
5. Mark the starting vertex as visited.
6. Display the vertex.
7. Visit all adjacent unvisited vertices recursively.
8. Repeat until all reachable vertices are visited.
9. Stop.

Program

```
#include <iostream>
using namespace std;

int graph[20][20], visited[20], n;

void DFS(int v)
{
   visited[v] = 1;
   cout << v << " ";

   for (int i = 0; i < n; i++)
   {
      if (graph[v][i] == 1 && visited[i] == 0)
      {
         DFS(i);
      }
   }
}

int main()
{
   int start;

   cout << "Enter number of vertices: ";
   cin >> n;

   cout << "Enter adjacency matrix:\n";
```

```
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cin >> graph[i][j];
        }
    }

    cout << "Enter starting vertex: ";
    cin >> start;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    cout << "DFS Traversal: ";
    DFS(start);
    cout << endl;

    return 0;
}
```

Output

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 1
1 0 0 0 0
0 1 0 0 0
0 1 0 0 0

Enter starting vertex: 0
DFS Traversal: 0 1 3 4 2
```

Result

Thus, the C++ program for Depth First Search (DFS) graph traversal was implemented and executed successfully.

EX.NO: 11 a	MINIMUM SPANNING TREE AND SHORTEST PATH ALGORITHMS
DATE:	IMPLEMENTATION OF MINIMUM SPANNING TREE ALGORITHMS KRUSKAL'S ALGORITHM

Aim

To write a C++ program to find the Minimum Spanning Tree (MST) of a weighted graph using Kruskal's Algorithm.

Algorithm

1. Start.
2. Read the number of vertices and the cost adjacency matrix.
3. Sort all edges based on their weights.
4. Select the edge with the minimum weight.
5. Check whether adding the edge forms a cycle.
6. If no cycle is formed, include the edge in the MST.
7. Repeat Steps 4–6 until (V - 1) edges are selected.
8. Calculate the total minimum cost.
9. Display the MST edges and cost.
10. Stop.

Program

```
#include  <iostream>

using namespace std;

int parent[20];

int find(int i)

{

   while (parent[i])

      i = parent[i];

   return i;

}

int uni(int i, int j)

{

   if (i != j)

   {

      parent[j] = i;

      return 1;

   }
```

```
    return 0;
}
int main()
{
    int n, cost[20][20];
    int mincost = 0, ne = 1;
    int a, b, u, v, min;
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter cost adjacency matrix:\n";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cin >> cost[i][j];
            if (cost[i][j] == 0)
                cost[i][j] = 999;
        }
    }
    cout << "\nEdges in MST:\n";
    while (ne < n)
    {
        min = 999;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
```

```
            }
        }
    }
    u = find(u);
    v = find(v);
    if (uni(u, v))
    {
        cout << ne++ << ". Edge (" << a << "," << b << ") = " << min << endl;
        mincost += min;
    }
    cost[a][b] = cost[b][a] = 999;
  }
  cout << "\nMinimum Cost = " << mincost << endl;
  return 0;
}
```

Output

Enter number of vertices: 4
Enter cost adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0

Edges in MST:

1. Edge (0,1) = 2 2.
Edge (1,2) = 3
3. Edge (2,3) = 5
Minimum Cost = 10

Result

Thus, the C++ program for finding the Minimum Spanning Tree using Kruskal's Algorithm was implemented and executed successfully.

EX.NO: 11 b	MINIMUM SPANNING TREE AND SHORTEST PATH ALGORITHMS
DATE:	IMPLEMENTATION OF MINIMUM SPANNING TREE ALGORITHMS PRIM'S ALGORITHM

Aim

To write a C++ program to find the Minimum Spanning Tree (MST) of a weighted graph using Prim's Algorithm.

Algorithm

1. Start.
2. Read the number of vertices and the cost adjacency matrix.
3. Replace all zero values (except diagonal) with infinity (999).
4. Mark the first vertex as visited.
5. Find the minimum cost edge connecting a visited vertex to an unvisited vertex.
6. Add the selected edge to the MST.
7. Mark the new vertex as visited.
8. Add the edge cost to the total minimum cost.
9. Repeat Steps 5–8 until (n-1) edges are selected.
10. Display the edges in the MST and the minimum cost.
11. Stop.

Program

```
#include <iostream>
using namespace std;

int main()
{
   int n, cost[10][10];
   int visited[10] = {0};
   int min, mincost = 0;
   int a = 0, b = 0, u = 0, v = 0, ne = 1;

   cout << "Enter number of vertices: ";
   cin >> n;

   cout << "Enter cost adjacency matrix:\n";
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         cin >> cost[i][j];
         if (cost[i][j] == 0)
            cost[i][j] = 999;
      }
   }
```

```
    visited[0] = 1;

    cout << "\nEdges in MST:\n";
    while (ne < n)
    {
        min = 999;

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (cost[i][j] < min)
                {
                    if (visited[i] != 0)
                    {
                        min = cost[i][j];
                        a = u = i;
                        b = v = j;
                    }
                }
            }
        }

        if (visited[u] == 0 || visited[v] == 0)
        {
            cout << ne << ". Edge (" << a << "," << b << ") = " << min << endl;
            mincost += min;
            visited[b] = 1;
            ne++;
        }

        cost[a][b] = cost[b][a] = 999;
    }

    cout << "\nMinimum Cost = " << mincost << endl;
    return 0;
}
```

Output

```
Enter number of vertices: 5
Enter cost adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in MST:

1. Edge (0,1) = 2 2.
Edge (1,2) = 3 3.
Edge (1,4) = 5
4. Edge (0,3) = 6

Minimum Cost = 16
```

Result

Thus, the C++ program for finding the Minimum Spanning Tree using Prim's Algorithm was implemented and executed successfully.

EX.NO: 11 C	MINIMUM SPANNING TREE AND SHORTEST PATH ALGORITHMS
DATE:	IMPLEMENTATION OF SHORTEST PATH ALGORITHMS DIJKSTRA'S ALGORITHM

Aim

To write a C++ program to find the shortest path from a source vertex to all other vertices in a weighted graph using Dijkstra's Algorithm.

Algorithm

1. Start.
2. Read the number of vertices and the adjacency matrix of the graph.
3. Read the source vertex.
4. Initialize:
    - Distance of source vertex as 0.
    - Distance of all other vertices as infinity.
    - Mark all vertices as unvisited.
5. Select the unvisited vertex with the minimum distance.
6. Mark it as visited.
7. Update the distances of all adjacent vertices.
8. Repeat Steps 5–7 until all vertices are visited.
9. Display the shortest distances from the source vertex.
10. Stop.

Program

```
#include  <iostream>
using namespace std;
#define V 10
#define INF 99999
int minDistance(int dist[], bool visited[], int n)
{
   int min = INF, min_index = 0;
   for (int v = 0; v < n; v++)
   {
      if (!visited[v] && dist[v] <= min)
      {
         min = dist[v];
         min_index = v;
      }
   }
   return min_index;
}
void dijkstra(int graph[V][V], int n, int src)
{
   int dist[V];
   bool visited[V];
```

```
    for (int i = 0; i < n; i++)
    {
        dist[i] = INF;
        visited[i] = false;
    }
    dist[src] = 0;
    for (int count = 0; count < n - 1; count++)
    {
        int u = minDistance(dist, visited, n);
        visited[u] = true;
        for (int v = 0; v < n; v++)
        {
            if (!visited[v] && graph[u][v] && dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v])
            {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    cout << "\nVertex\tDistance from Source\n";
    for (int i = 0; i < n; i++)
    {
        cout << i << "\t" << dist[i] << endl;
    }
}
int main()
{
    int n, src;
    int graph[V][V];
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter adjacency matrix:\n";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cin >> graph[i][j];
        }
    }
    cout << "Enter source vertex: ";
    cin >> src;

    dijkstra(graph, n, src);
    return 0;
}
```

Output

```
Enter number of vertices: 5
Enter adjacency matrix:
0  10  0  30  100
10  0  50  0  0
0  50  0  20  10
30  0  20  0  60
100 0  10  60  0
Enter source vertex: 0


Vertex  Distance from Source
0       0
1       10
2       50
3       30
4       60
```

Result

Thus, the C++ program to find the shortest path from a source vertex to all other vertices using Dijkstra's Algorithm was implemented and executed successfully.